

RAPTOR 递归摘要检索技术指南

一、长文档检索面临的真实困境

传统检索增强生成系统处理长文档时，通常采用固定长度切分策略，把一篇文档切成若干互不重叠或轻微重叠的文本块，再对每个文本块单独建立向量索引。这种做法在短查询、事实型问题上表现尚可，一旦用户提出需要跨段落综合的宏观问题，效果就会急剧下降。原因并不复杂：一个只包含两百字的文本块，天然无法回答需要通读全文才能归纳的问题。检索器会在局部相似度最高的几个块中挑选答案，而这些块可能只是某个章节的开头，缺少结论，缺少上下文，也缺少把多个章节联系起来的那条主线。

更麻烦的是，固定长度切分往往会切断语义单元。一个完整的论证可能横跨三个块，结论落在第四个块的开头。用户查询命中了前三个块中的某一个，模型拿到的却是一段没有结论的残缺论证。为了缓解这个问题，工程上常见的补救手段是加大块长度或者增加重叠窗口。加大块长度会让单个块的语义变得混杂，向量表示被稀释，检索精度反而下降；增加重叠则会造成大量重复内容，存储成本、嵌入成本和后续检索的去重成本都会上升。这构成了一组很难同时满足的矛盾。

另一个被低估的问题是层级缺失。人类阅读长文档时是分层理解的：先看目录，再读章节，最后才落到具体段落。检索系统如果只提供段落级别的入口，就等于取消了目录这一层。用户提出宏观问题时，系统手里没有与之匹配的抽象层次可供检索，只能退而求其次地返回几个细节片段。

二、RAPTOR 的核心思想

RAPTOR 这个名字来自 Recursive Abstractive Processing for Tree-Organized Retrieval，其核心主张是：与其只在最细粒度上建索引，不如把文档组织成一棵语义树，让不同抽象层次的内容都能被检索到。树的叶子是原始文本块，越往上层走，节点所覆盖的原文范围越大，摘要的抽象程度越高，最终收敛到一个覆盖全文的根节点。

构建过程是一个自底向上的递归循环。先把文档切成叶子块并向量化；然后对叶子块的向量做降维，在降维后的空间里做聚类，把语义相近的块分到同一簇；接着对每一簇调用大语言模型生成一段摘要，这个摘要就成为该簇的父节点；父节点同样被向量化，与同层的其他父节点一起进入下一轮聚类与摘要，直到某一层只剩下一个节点，这棵树就收敛了。

这样组织之后，检索可以在任意层级发生。用户问一个具体事实，命中的可能是叶子块；用户问一个章节主旨，命中的可能是中间层摘要；用户问整篇文档在讲什么，命中的可能是靠近根节点的摘要。系统返回的结果天然携带层级信息，调用方可以根据层级决定后续如何使用这些内容。

RAPTOR 的价值不只是多了一层索引。它实际上把原本隐藏在原文结构里的语义层次显式地提取了出来，并且用向量空间中的邻近关系来表达这种层次。当原文本身结构混乱、缺少小标题、或者是由多个来源拼接而成时，这种自动提取的层次尤其有价值。

三、分块策略的设计取舍

分块是整条流水线的第一道工序，它的质量直接决定后续所有环节的上限。如果分块把语义单元切碎，再好的聚类和摘要算法也无力回天。

工程上常见三种策略。第一种是固定长度分块，按字符数或词数机械切分，实现简单、行为可预测，但无视语义边界。第二种是递归字符分块，按优先级依次尝试段落、换行、句号、逗号等分隔符，尽量在自然边界处断开，是通用场景下最稳妥的默认选择。第三种是结构感知分块，直接解析 Markdown 标题层级、Word 样式、PDF 的章节结构，在真正的一级或二级标题处切分，效果最好但依赖文档格式的规范性。

块长度需要结合嵌入模型的能力来定。多数中英文嵌入模型的有效上下文在数百到上千个中文字符之间，超过之后语义会被平均化，向量区分度下降。实践经验是中文文档单块取五百到八百字比较平衡，既能承载一个完整论点，又不至于让向量失焦。重叠度一般取块长度的百分之十到百分之二十，用于防止语义单元被硬切断，但重叠过高会显著增加嵌入调用量和存储占用。

分块的另一个隐性要求是元数据完整。每个块必须记录它来自哪个文档、在文档中的顺序号、字符数，以及原始字符偏移量。偏移量在需要回溯原文定位时不可或缺，顺序号则决定了检索结果能否按原文顺序重排。缺少这些字段，后续做可解释的召回展示和召回率评估都会非常困难。

四、向量化与嵌入模型的选择

文本块必须转成向量才能参与语义检索。嵌入模型把一段文本映射到一个高维实数向量，语义相近的文本在这个空间中距离更近。这个性质是整套系统赖以工作的基础假设。

选择嵌入模型时要权衡三件事：向量维度、语言覆盖、以及调用成本。维度越高，理论上能表达的信息越多，但存储和计算成本随之上升，而且收益存在明显的边际递减。主流中文嵌入模型多为一千零二十四维或一千五百三十六维。当模型支持自定义输出维度时，一般选择与已有基础设施匹配的维度，避免为了追高维度而改动表结构。

调用方式上有两个工程细节值得注意。第一是批量嵌入。绝大多数兼容接口都支持一次传入多个文本，返回同样数量的向量，这能显著摊薄网络往返开销。第二是缓存。重复的文本块其实很常见，尤其在文档被重复导入或者多个知识库共享同一份资料时，按文本内容哈希做一层缓存可以直接省下可观的调用量。

还有一个容易踩的坑是向量归一化。使用余弦距离时，向量是否归一化不影响排序结果，因为余弦相似度本身就做了长度归一。但如果后续混用了内积或欧氏距离，归一化状态不一致会导致分数不可比。统一的工程做法是在写入前保证所有向量处于同一归一化状态，并在检索时使用与之匹配的距离算子。

五、UMAP 降维的原理与作用

把一千多个维度的向量直接拿去做聚类，会遇到所谓的维度灾难：在高维空间中，距离度量变得不再可靠，点与点之间的距离趋于集中，聚类算法很难分辨出真实的结构。降维的目的就是在保留语义邻近关系的前提下，把数据压到低维空间，让聚类算法能够工作。

UMAP 全称统一流形近似与投影，它的基本假设是数据点分布在一个嵌入高维空间的低维流形上。算法先用最近邻图构建一个模糊的拓扑表示，再通过优化低维布局，让低维空间的拓扑结构与高维空间尽可能一致。相比主成分分析这类线性方法，UMAP 能保留非线性结构，因此对语义数据通常更合适。

UMAP 有两个关键参数。`n_neighbors` 控制局部与全局的平衡：取值小的时候，算法更关注局部邻域，簇会变得细碎，容易切出很多小簇；取值大的时候，算法倾向把握全局结构，簇会更大更连贯。中文文档场景下，十几到三十之间是常用的起点。`min_dist` 控制低维空间中点与点之间的最小距离：值越小，点越

倾向于紧密成团，簇内更紧凑，适合聚类；值越大，布局越松散，更适合可视化展示。做聚类时通常把 `min_dist` 压得比较低。

必须提醒的是，UMAP 对样本量有要求。当样本只有几个或十几个时，最近邻图的构建本身就失去了意义，输出的坐标不可信，甚至可能直接抛异常。因此整条流水线必须做兜底：当某一层的节点数低于某个阈值时，跳过降维，直接在原始向量空间聚类，或者干脆不再分裂、把所有节点合并成一个父节点。这个兜底逻辑不是可选项，而是保证递归能够收敛的必要条件。

UMAP 还带有随机性，同样的输入在不同运行中可能得到略有差异的布局。为了保证同一份文档多次构建出的树结构稳定，应当显式固定随机种子。这在调试聚类参数时尤其重要，否则参数效果的对比会被随机波动淹没。

六、GMM 聚类与簇数选择

高斯混合模型把数据看作由若干个高斯分布混合生成，每个高斯分量对应一个簇。与 K 均值这类硬聚类不同，GMM 输出的是每个点属于各个簇的概率，属于软聚类。这个特性对 RAPTOR 很有价值：一个横跨两个主题的中间段落，可以被两个簇同时以一定概率吸收，从而出现在两段不同的摘要里，避免信息被硬性割裂。

GMM 需要预先指定簇数，或者给出一个上限让算法自己去选。常见做法是让算法在给定范围内尝试不同簇数，用贝叶斯信息准则这类指标挑最优，BIC 在模型拟合优度与复杂度之间做惩罚权衡，能避免选出过多琐碎的簇。工程实现里必须同时设定最小簇数与最大簇数，最小簇数保证不会只切出一个簇，最大簇数防止在大规模数据上把 GMM 变成一个组合爆炸的搜索过程。

簇数上限应当与当前层的节点数挂钩。如果当前层只有二十个节点，允许分成十五个簇显然荒唐，摘要会退化成对单个块的复述，失去压缩意义。一个实用的经验规则是把最大簇数限制在节点数的平方根附近，同时设置一个绝对上限，例如十个或十五个。这个规则让不同规模的数据都能得到合理粒度。

聚类结果还需要做质量兜底。如果某个簇只包含一个节点，对它做摘要就是无意义的开销，应当直接把该节点提升为父节点或者并入相邻簇。如果所有节点被分到同一个簇，说明这一层已经没有可分裂的语义结构，递归应当终止。

七、递归摘要与树结构

摘要环节是 RAPTOR 与前缀树、固定层级索引等方案最本质的区别。它不是对原文做机械摘取，而是调用大语言模型做真正的语义压缩。这一步的质量决定了中间层和根层的检索效果。

摘要提示词必须严格约束。大语言模型天然倾向于补充背景、给出建议、做价值判断，这些行为在摘要场景下都是污染：一旦摘要里出现了原文没有的信息，上层节点就变成了幻觉源，检索到它反而会误导下游。因此提示词必须明确要求仅基于所提供的文本块进行总结，禁止添加任何未在原文中出现的信息，只做信息压缩，不添加新事实。同时要求保留关键实体、数字和专有名词，因为这些往往是用户查询的锚点。

递归的终止条件需要同时考虑两个维度。一是结构维度：当某一层只剩一个节点时，树的根已经形成，可以停止。二是深度维度：当达到预设的最大层级时，无论还剩多少节点都停止向上构建，把当前层作为顶层。设置深度上限的动机是成本控制：每往上走一层都要付出一次聚类和若干次摘要调用，对于超长文档

，不加限制的递归可能产生十几层，收益却早已饱和。

树的每个节点都要完整落库。需要的字段包括层级、父节点标识、节点类型、摘要内容或原始块内容、向量表示，以及该节点覆盖的叶子块范围。覆盖范围这个字段常被忽略，但它对可解释性至关重要：当用户检索到某个中间层摘要时，系统可以据此回答这个结论来自原文的哪几段，从而支持溯源。

八、三种检索模式的差异

叶子索引只能回答细节问题，树索引可以回答宏观问题，两者结合才是完整的检索能力。工程上通常提供三种模式供对比与调优。

纯向量检索依赖语义相似度，擅长处理同义表达和概念性问题。用户用完全不同的措辞描述同一个意思时，向量检索依然能找到相关内容，这是关键词检索做不到的。它的短板是对精确匹配不敏感：专有名词、编号、代码标识符这类信息在向量空间里往往区分度不足，容易漏召回。

纯 BM25 检索是经典的概率检索模型，基于词频与逆文档频率打分，并做了文档长度归一化。它擅长精确匹配，查询里出现什么词就去找什么词，对罕见词和专业术语非常敏感。它的短板是缺乏语义泛化能力，用户换个说法就完全找不到。中文场景下 BM25 的效果高度依赖分词质量，分词粒度不当会同时损害召回和排序。

混合检索把两路结果融合，兼顾语义泛化与精确匹配。融合策略里最常用的是倒数排名融合，简称 RRF。它的做法是对每一路结果，按排名位置计算一个分数，排名第一得满分，越靠后分数越低，然后把同一个文档在各路中的分数加起来重新排序。RRF 的妙处在于只依赖排名而不依赖原始分数，因此不需要解决向量相似度与 BM25 分数之间的量纲不可比问题。公式里有一个平滑常数，通常取六十，它的作用是削弱排名靠前与靠后之间的分数差距，避免某一路的头部结果垄断最终排序。这个常数越小，头部排名的优势越被放大；越大，各路结果越趋于平均。

融合权重也应当可调。直接叠加两路分数时，可以按权重比例缩放；也可以在召回阶段就按权重决定每路取多少个候选。无论采用哪种，权重与平滑常数都必须暴露成可调参数，因为最优取值高度依赖语料：偏技术手册的语料更依赖关键词匹配，偏叙述性的语料则更依赖语义匹配。

九、评估指标的定义与意义

没有量化评估，参数调优就是盲猜。RAPTOR 场景下最常用的三个指标是召回率、命中率和平均倒数排名。

召回率衡量的是查全能力。给定一组测试查询，每个查询标注了若干应当被命中的文本块，召回率等于被成功召回的期望块数量占期望块总数的比例，通常考察前 K 个结果，记作 Recall@K。它回答的问题是：该找回来的东西，找回来了多少。

命中率是一个更宽松的指标，只要前 K 个结果里出现了任意一个期望块，这个查询就算命中，命中率等于命中的查询数占总查询数的比例，记作 Hit Rate@K。在期望块有多个、但用户其实只关心其中一个的场景下，命中率比召回率更贴近真实体验。

平均倒数排名关注的是排序质量。对每个查询，找到第一个命中的期望块，取它的排名倒数值，即排名为

第几就取几分之一，然后对所有查询求平均，记作 MRR。它惩罚把正确结果排到很后面的情况，衡量的是首次命中来得有多快。

这三个指标必须一起看。召回率高但平均倒数排名低，说明内容找回来了但排得很靠后，用户体验依然糟糕；命中率高但召回率低，说明系统只擅长回答那种期望块唯一的问题。此外，测试集里期望块的标注质量决定了评估结论的可信度，标注本身需要人工确认，不能由系统自动生成，否则评估就变成了自我循环。

十、工程落地中的异步与可观测性

整条流水线的耗时分布很不均匀。文档解析通常在毫秒到秒级，向量化取决于块数量与批处理效率，聚类在万级数据以下很快，真正的大头是摘要环节：每调用一次大语言模型都有网络往返和生成时间，一棵树可能有几十个中间节点，串行执行会让用户等上几分钟。

因此导入、分块、向量化、建树这些阶段必须做成异步任务。请求进来立刻返回任务标识，前端轮询任务进度，把每一步的状态和百分比展示给用户。任务状态必须持久化，不能只放在内存里：进程重启、异常中断之后再查任务，应当还能看到它停在哪一步、失败原因是什么。持久化的状态也是断点续跑的基础。

可观测性方面，每一次检索都应当留下日志。需要记录的字段包括原始查询、检索模式、全部参数取值、各路召回的候选数量、最终返回数量，以及端到端耗时。这些日志在排查「为什么这次没召回」时价值极高，也是评估不同参数组合效果的原始数据来源。日志表要注意控制体积，可以提供按时间清理的策略。

最后是数据不可变原则。已导入的文档正文不应允许修改，因为修改会让已建好的摘要树与原文不再对应，树上任何一个中间节点都可能引用了已经不存在的原文。合理的做法是只允许禁用：禁用后该文档的块不参与检索、也不参与后续建树，但数据完整保留，随时可以恢复。需要更正内容时，应当重新导入一份新文档，而不是就地修改。这条原则看起来限制了灵活性，实际上保护了整个索引的一致性和可追溯性。